

6.4.2 Case-based reasoning

Case-based reasoning (CBR) classifiers use a database of problem solutions to solve new problems. Unlike k -nearest-neighbor classifiers, which store training tuples as points in Euclidean space, CBR stores the tuples or “cases” for problem solving as complex symbolic descriptions. Business applications of CBR include problem resolution for customer service help desks, where cases describe product-related diagnostic problems. CBR has also been applied to areas, such as engineering and law, where cases are either technical designs or legal rulings in the common law system, respectively. Medical education is another area for CBR, where patient case histories and treatments are used to help diagnose and treat new patients.

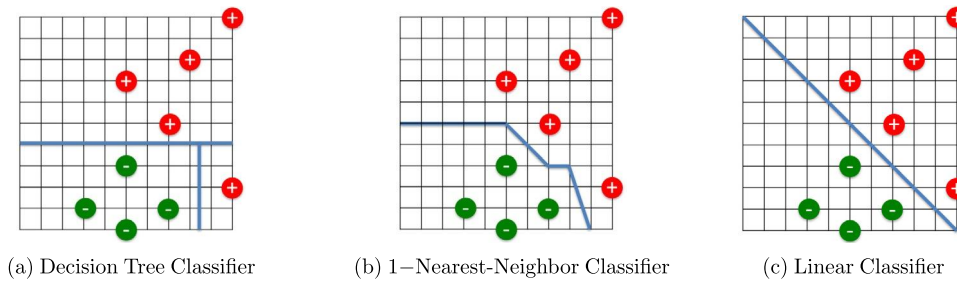
When given a new case to classify, a case-based reasoner will first check if an identical training case exists. If one is found, then the accompanying solution to that case is returned. If no identical case is found, then the case-based reasoner will search for training cases having components that are similar to those of the new case. Conceptually, these training cases may be considered as neighbors of the new case. If cases are represented as graphs, this involves searching for subgraphs that are similar to subgraphs within the new case. The case-based reasoner tries to combine the solutions of the neighboring training cases to propose a solution for the new case. If incompatibilities arise with the individual solutions, then backtracking to search for other solutions may be necessary. The case-based reasoner may employ background knowledge and problem-solving strategies to propose a feasible combined solution.

Key challenges in case-based reasoning include finding a good similarity metric (e.g., for matching subgraphs) and suitable methods for combining solutions. Other challenges include the selection of salient features for indexing training cases and the development of efficient indexing techniques. A trade-off between accuracy and efficiency evolves as the number of stored cases becomes very large. As this number increases, the case-based reasoner becomes more intelligent. After a certain point, however, the system’s efficiency will suffer as the time required to search for and process relevant cases increases. As with nearest-neighbor classifiers, one solution is to edit the training database. Cases that are redundant or those that have not proved useful may be discarded for the sake of improved performance. These decisions, however, are not clear-cut, and their automation remains an active area of research.

6.5 Linear classifiers

So far, we have learned a few classifiers which are capable of generating complex decision boundaries. For example, a decision tree classifier might output a hyperrectangular-shaped decision boundary (Fig. 6.10(a)), and a k -nearest-neighbor classifier might output a hyperpolygonal-shaped decision boundary (Fig. 6.10(b)). However, what about a simple, linear decision boundary? For the example in Fig. 6.10, intuitively, a linear decision boundary (the straight line in Fig. 6.10(c)) is (almost) as good as decision tree classifiers and k -nearest-neighbor classifier in separating the positive training tuples from the negative training ones. Yet, such a linear decision boundary might offer additional advantages, such as efficient computation for training the classifier, better generalization performance, and better interpretability.

In this section, we introduce basic techniques to learn such linear classifiers. We will start with **linear regression**, which forms the basis for linear classifiers. Then, we will introduce two linear classi-

**FIGURE 6.10**

Decision boundaries by different classifiers. Note that this example is *linearly separable*, meaning that a linear classifier (c) can perfectly separate all the positive training tuples from all the negative training tuples. If the training set is *linearly inseparable*, we could still use a linear classifier, at the expense that some training tuples are on the ‘wrong’ side of the decision boundary. In Chapter 7, we will introduce techniques (e.g., support vector machines) to handle linearly inseparable case.

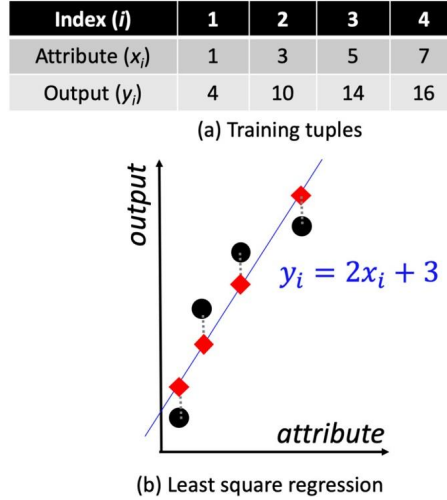
fiers, including (1) **perception**, which is one of the earliest linear classifiers, and (2) **logistic regression** which is one of the most widely used linear classifiers. Additional linear classifiers will be introduced in Chapter 7, such as linear support vector machines.

6.5.1 Linear regression

Linear regression is a statistical technique that predicts a continuous value based on one or more independent attributes. For example, we might want to predict the housing price based on the living area or to predict the future income of a student based on which college she attended, in which major and the overall GPA, etc. Since linear regression aims to predict a *continuous* value, it cannot be directly applied to the classification task, where the output is a categorical variable. Nonetheless, the core techniques in linear regression form the basis of linear classifiers. Therefore, let us first briefly introduce linear regression.

Suppose we have n tuples, each of which is represented by p attributes $x_i = (x_{i,1}, \dots, x_{i,p})^T$ and a continuous output value y_i ($i = 1, \dots, n$). In linear regression, we want to learn a linear function that maps the p input attributes x_i to the output variable y_i , that is, $\hat{y}_i = w^T x_i + b = \sum_{j=1}^p w_j x_{i,j} + b$, where $\hat{y}_i$ is the predicted output value for the i th tuple, $w = (w_1, \dots, w_p)^T$ is a p -dimensional weight vector and b is the bias scalar. In other words, linear regression assumes that the output value is a linear weighted summation of the p input attribute values, offset by the bias scalar b . The entries in the weight vector w_j ($j = 1, \dots, p$) tell how important the corresponding attribute $x_{i,j}$ is in predicting the output variable $\hat{y}_i$. In the aforementioned examples, a linear regression model would assume that the housing prices are linearly correlated with the living area; the future income of a student can be predicted by a linear weighted combination of the college she attended, the major, and the overall GPA (plus a bias scalar b). If we know the weight vector w and the bias scalar b , we can make a prediction of the output value based on its p input attribute values.

“So, how can we determine the weight vector w and the bias scalar b ?” Intuitively, we want to learn the “best” weight vector w and the “best” bias scalar b from the training data, so that the lin-

**FIGURE 6.11**

An example of least square regression. (a) Four training tuples. (b) Scatter-plot of the training tuples (black dots) and least square regression model (the blue line). Red diamonds are the predicted output $\hat{y}_i$ ($i = 1, 2, 3, 4$) and dashed lines indicate the prediction errors ($|y_i - \hat{y}_i|$) of the corresponding training tuples.

ear regression model can make the “best” prediction. That is, the predicted value $\hat{y}_i = w^T x_i + b$ is as close as possible to the actual observed value y_i ($i = 1, \dots, n$). One of the most common linear regression methods is called **least square regression**, which aims to minimize the following loss function $L(w, b) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (w^T x_i + b))^2$. Therefore the best weight vector w and the bias scalar b are the ones that minimize the loss function $L(w, b)$, which measures the sum of the squared difference between the predicted output value $\hat{y}_i$ and the actual observed value y_i . For example, if there is only one input attribute (i.e., $p = 1$), the optimal weight $w = \frac{\sum_{i=1}^n x_i (y_i - \bar{y})}{\sum_{i=1}^n x_i^2 - \frac{1}{n} (\sum_{i=1}^n x_i)^2}$ and the optimal bias scalar $b = \frac{1}{n} \sum_{i=1}^n (y_i - w x_i)$, where $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ is the average observed output value among all n training tuples.

Example 6.7. Let us look at an example of least square regression in Fig. 6.11. There are four training tuples, each represented by a single-dimensional attribute x_i and an output variable y_i ($i = 1, 2, 3, 4$). We want to find least square regression model $y = wx + b$ that predicts the output y based on the input attribute x . We use the two equations mentioned above to find the optimal weight w and the optimal bias scalar b . We first find the optimal weight w as follows. The average output of four training tuples is $\bar{y} = (y_1 + y_2 + y_3 + y_4)/4 = (4 + 10 + 14 + 16)/4 = 11$. Therefore we have that $\sum_{i=1}^4 x_i (y_i - \bar{y}) = 1(4 - 11) + 3(10 - 11) + 5(14 - 11) + 7(16 - 11) = 40$. In the meanwhile, we have that $\sum_{i=1}^4 x_i^2 = 1^2 + 3^2 + 5^2 + 7^2 = 84$ and $1/4(\sum_{i=1}^4 x_i)^2 = (1 + 3 + 5 + 7)^2/4 = 64$. Therefore the optimal weight $w = \frac{\sum_{i=1}^n x_i (y_i - \bar{y})}{\sum_{i=1}^n x_i^2 - \frac{1}{n} (\sum_{i=1}^n x_i)^2} = \frac{40}{84 - 64} = 2$. Based on the optimal weight w , the optimal bias scalar $b = \frac{\sum_{i=1}^4 (y_i - w x_i)}{4} = \frac{(4 - 2 \times 1) + (10 - 2 \times 3) + (14 - 2 \times 5) + (16 - 2 \times 7)}{4} = 3$. □

“But, what if there are multiple p ($p > 1$) attributes?” In this case (which is called **multilinear regression**), let us first change our notation a little bit. We assume there is an additional “dummy” attribute which always takes the value of 1 for any tuple. Let the weight for this dummy attribute be w_0 . Then the overall weight vector $w = (w_0, w_1, \dots, w_p)$ and the new input attribute vector $x_i = (1, x_{i,1}, \dots, x_{i,p})$ are both $(p + 1)$ -dimensional vectors. The multilinear regression model can be re-written as $\hat{y}_i = w^T x_i = w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}$. We use the same loss function as before, that is, $L(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (w^T x_i))^2$. It turns out the optimal weight vector w can be computed as $w = (XX^T)^{-1}Xy$, where $X = [x_1, x_2, \dots, x_n]$ is a $(p + 1) \times n$ matrix, and $y = [y_1, \dots, y_n]^T$ is an $n \times 1$ vector. (How to derive the closed form solutions for single linear regression as well as multilinear regression are left as exercises.)

In least square regression, we measure the “goodness” of the learned regression model by the sum of the squared difference between predicted and actual output values. The squared loss might be sensitive to the outliers in the training set. In *robust regression*, it uses alternative loss functions that are less sensitive to such outliers. For example, the Huber method in robust regression uses the following loss: $L(w) = \sum_{i=1}^n l_H(y_i - \hat{y}_i)$, where $l_H(y_i - \hat{y}_i) = (y_i - \hat{y}_i)^2$ if $|y_i - \hat{y}_i| < \theta$, $l_H(y_i - \hat{y}_i) = 2\theta|y_i - \hat{y}_i| - \theta^2$ otherwise, and $\theta > 0$ is a user-specified parameter. Notice that the optimal weight vector w for multilinear regression involves a matrix inverse (i.e., $(XX^T)^{-1}$). In case $p > n$ (i.e., the number of attributes is more than the number of training tuples), such a matrix inverse does not exist. An effective way to address this issue is to introduce a *regularization term* regarding the norm of the weight vector w . For example, if we use l_2 norm of the weight vector w , the corresponding regression model is called Ridge regression; if we use l_1 norm of the weight vector w instead, the corresponding regression model is called Lasso regression which often learns a *sparse* weight vector. This means that some entries of the learned weight vector w are zeros, which indicates that those attributes are not used in the regression model. In Section 7.1, we will use Lasso regression for feature selection.

6.5.2 Perceptron: turning linear regression to classification

“How can we modify a linear regression model to perform classification task?” Suppose we have a binary classification task.¹⁰ The output value y_i for a given tuple is a binary variable: $y_i = +1$ indicates the i th tuple is a positive tuple (e.g., *buy computer*) and $y_i = 0$ indicates the i th tuple is a negative one (e.g., *not buy computer*). One way to modify the linear regression model for such a binary classification task is to use the *sign* of the output of the linear regression model as the predicted class label, that is, $\hat{y}_i = \text{sign}(w^T x_i)$, where $\hat{y}_i$ is the predicted class label for i th tuple, $\text{sign}(z) = 1$ if $z > 0$ and $\text{sign}(z) = 0$ otherwise. Notice that we use the same notation as multilinear regression where we have introduced a “dummy” attribute which always takes the value of 1 for any tuple. Therefore if we know the weight vector w , we can use it to predict the class label of a given tuple as follows. We compute a linear combination of the attribute values of the given tuple, weighted by the corresponding entries of the weight vector w . If the resulting value of such a linear combination is positive, we predict that the given tuple is a positive tuple. Otherwise, we predict that it is a negative one.

“How can we find the optimal weight vector w from a set of training tuples?” The classic learning algorithm to train a perceptron is as follows. We start with an initial guess of the weight vector w (e.g.,

¹⁰ For both perceptron and logistic regression classifiers that we will introduce next, we focus on binary classification task. However, the techniques we introduce can be generalized to handle multiclass classification task for both classifiers.

we can simply set $w = 0$). Then, the learning algorithm will iterate until it converges, or the maximum iteration number or some other preset stopping criteria are met. In each iteration, we do the following for each training tuple x_i . We try to predict the class label of x_i using the current weight vector w , that is, $\hat{y}_i = \text{sign}(w^T x_i)$. If the prediction is correct (i.e., $\hat{y}_i = y_i$), we do nothing about the weight vector. However, if the prediction is incorrect (i.e., $\hat{y}_i \neq y_i$), we update the current weight vector in one of the following two ways. If $y_i = +1$ (i.e., the i th tuple is a positive tuple, but the current classifier predicts it is a negative tuple), we update weight vector as $w \leftarrow w + \eta x_i$. If $y_i = -1$ (i.e., the i th tuple is a negative tuple, which is wrongly predicted by the current classifier as a positive tuple), we update weight vector as $w \leftarrow w - \eta x_i$, where $\eta > 0$ is the user-specified learning rate. So, the intuition is that in each iteration of the training process, the algorithm will focus on those wrongly predicted training tuples by the current weight vector w . If the wrongly predicted training tuple x_i is a positive tuple, we update the weight vector w by moving it *towards* the attribute vector x_i of this training tuple (i.e., $w \leftarrow w + \eta x_i$). On the other hand, if the wrongly predicted training tuple x_i is a negative tuple, we update the weight vector w by moving it *away from* the attribute vector x_i of this training tuple (i.e., $w \leftarrow w - \eta x_i$).

Example 6.8. Let us look at an example in Fig. 6.12 for training a perceptron classifier. In Fig. 6.12, we assume the bias $w_0 = 0$ for illustration clarity. Fig. 6.12(a) (left) shows the current decision boundary and the weight vector w , where two training tuples are wrongly classified, including a positive tuple x_1 and a negative tuple x_8 . Therefore only these two tuples are used to update the weight vector in the current iteration, that is, $w \leftarrow w + \eta x_1 - \eta x_8$. The updated weight vector w and the corresponding decision boundary are shown in Fig. 6.12(b) (right), where all training tuples are correctly classified.

□

“How effective is the perceptron learning algorithm?” If the training tuples are linearly separable (e.g., the example in Fig. 6.12), the perceptron algorithm is guaranteed to find a weight vector (i.e., a hyperplane decision boundary) that perfectly separates all the positive training tuples from all the negative training tuples. However, if the training tuples are not linearly separable, this algorithm will fail to converge.

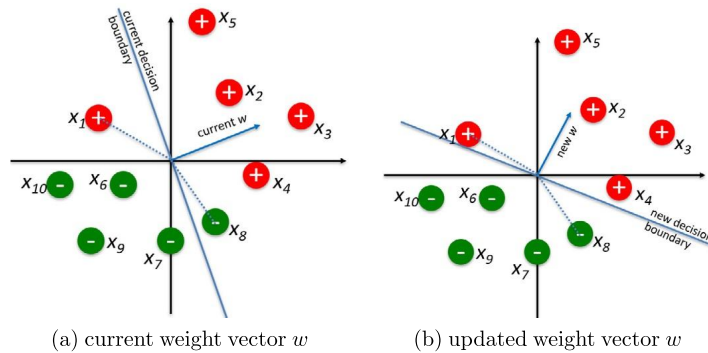


FIGURE 6.12

Training a perceptron classifier.

Perceptron, one of the earliest linear classifiers, was first invented back in 1958. It can also be used as a building block (called a “neuron”) in deep neural networks that will be introduced in Chapter 10.

6.5.3 Logistic regression

Perceptron that we have just introduced in the previous section is capable of predicting the binary class label of a given tuple. However, can we also tell how confident such a prediction is? Again, let us consider a binary classification task, and we assume that there are two possible class labels, that is, $y = 1$ for a positive tuple and $y = 0$ for a negative tuple. Recall that in (naïve) Bayes classifier, we can estimate the posterior probability $P(y_i = 1|x_i)$, which can be directly used to indicate how confident the predicted classification result is. For example, if $P(y_i = 1|x_i)$ is close to 1, the classifier is highly confident that the tuple x_i is a positive example.

How can we make a linear classifier not only predict which class label a tuple has, but also tell how confident it is in making such a prediction? An effective way to this end is via **logistic regression** classifier. Let us first introduce an important function called **sigmoid** function, which is defined as $\sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z}$. From Fig. 6.13, we can see that the sigmoid function maps a real number in $(-\infty, +\infty)$ (i.e., the x-axis of Fig. 6.13) to an output value in the range of $(0, 1)$ (i.e., the y-axis of Fig. 6.13). Therefore if we leverage the sigmoid function to map the output of a linear regression model to a number between 0 and 1, we can interpret the mapping result as the posterior probability of observing a positive class label. This is exactly what logistic regression classifier tries to do!

Formally, we have $P(\hat{y}_i = 1|x_i, w) = \sigma(w^T x_i) = \frac{1}{1+e^{-w^T x_i}}$, where $\hat{y}_i$ is the predicted class label for the tuple with attributes x_i , and w is the weight vector. Notice that we have absorbed the bias term b into the weight vector w by introducing a dummy attribute to simplify the notation, as we did in the multilinear regression model and in perceptron. Naturally, if $P(\hat{y}_i = 1|x_i, w) > 0.5$, the classifier predicts that the tuple x_i is a positive tuple (i.e., $\hat{y}_i = 1$), otherwise, it predicts a negative tuple (i.e., $\hat{y}_i = 0$). This (details are left as an exercise) is equivalent to the following linear classifier: predict $\hat{y}_i = 1$ (i.e., positive tuple) if $w^T x_i > 0$, and predict $\hat{y}_i = 0$ (i.e., negative tuple) if $w^T x_i < 0$. Therefore if we know the weight vector w , the classification task for a given tuple is quite simple. That is, we

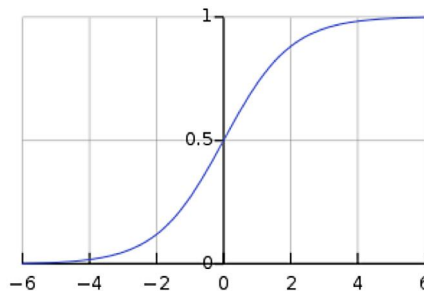


FIGURE 6.13

Illustration of sigmoid function. The sigmoid function “squashes” an input from a larger range $(-\infty, +\infty)$ to a smaller range $(0, 1)$. For this reason, sigmoid function is also called *squash function*. In Chapter 10, we will see other types of squash functions, which are called activation functions in the deep learning terminology.

only need to multiply the attribute vector x_i of the given tuple with the weight vector w , and then make a prediction based on the sign of $w^T x_i$. If $w^T x_i$ is a positive number, we predict that the given tuple is a positive tuple. Otherwise, we predict that it is a negative tuple.

“How can we determine the optimal weight vector w from a set of training tuples?” The classic method to train a logistic regression classifier (i.e., to determine the best weight vector w from the training set) is via *maximum likelihood estimation* (MLE). Again, let us assume there are n training tuples (x_i, y_i) ($i = 1, \dots, n$). Since we have a binary classification task, we can view the predicted class label $\hat{y}_i$ as a Bernoulli random variable, which can only take two possible values, including $P(\hat{y}_i = 1|x_i, w) = p_i$ and $P(\hat{y}_i = 0|x_i, w) = 1 - p_i$, where $p_i = \sigma(w^T x_i) = \frac{1}{1 + e^{-w^T x_i}}$ is determined by the sigmoid function and it describes the probability of observing a positive outcome for the predicted class label (i.e., $\hat{y}_i = 1$). Notice that the true class label y_i for the i th tuple is a binary variable. Therefore we have that $P(\hat{y}_i = y_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$. The maximum likelihood estimation method aims to solve the following optimization problem, which says that we should choose the best weight vector w that maximizes the likelihood of the training set. The intuition is that we want to find the optimal model parameter (i.e., the weight vector w) so that there is the highest “chance” (i.e., the likelihood or the probability) of observing the entire training set.

$$w^* = \operatorname{argmax}_w L(w) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i} = \prod_{i=1}^n \left(\frac{e^{w^T x_i}}{1 + e^{w^T x_i}} \right)^{y_i} \left(\frac{1}{1 + e^{w^T x_i}} \right)^{1-y_i} \quad (6.19)$$

“But, how can we develop an algorithm to solve this optimization problem to find the optimal weight vector w ?” First, we notice that the likelihood function $L(w)$ has many nonnegative terms that are multiplied with each other. In practice, it is often more convenient to work with the logarithm of such a complicated function. Thus we have the following equivalent optimization problem, where $l(w)$ is called the log likelihood

$$w^* = \operatorname{argmax}_w l(w) = \sum_{i=1}^n y_i x_i^T w - \log(1 + e^{w^T x_i}). \quad (6.20)$$

From the optimization perspective, the good news is that the log likelihood function in Eq. (6.20) is a strictly concave function, and therefore its maximum (the optimal solution) uniquely exists. However, the bad news is that the closed-form solution for the above optimization problem does not exist. In this case, a common strategy is to find the optimal solution w^* iteratively as follows. In each iteration, we try to improve the current weight vector w so that the objective function we wish to maximize (the log likelihood function $l(w)$) is improved most. In order to increase the current objective function $l(w)$ most, it turns out the best direction to update the current estimation of the weight vector w is to follow its *gradient*. This leads to the following algorithm to learn the optimal weight vector w^* from the training set. We start with an initial guess of the weight vector w (e.g., we can simply set $w = 0$). Then, the learning algorithm will iterate until it converges, or the maximum iteration number or some other preset stopping criteria are met. In each iteration, it updates the weight vector w as follows $w \leftarrow w + \eta \sum_{i=1}^n (y_i - P(\hat{y}_i = 1|x_i, w))x_i$, where $\eta > 0$ is the user-specified learning rate.

“So, what is the intuition of the above algorithm?” Let us analyze the impact of each training tuple (x_i, y_i) on updating the estimation of the weight vector w . We consider two situations depending on whether it is a positive tuple (i.e., $y_i = 1$) or a negative tuple (i.e., $y_i = 0$). For the former, the

impact of the given tuple on updating the weight vector w can be calculated as $w \leftarrow w + \eta(1 - P(\hat{y}_i = 1|x_i, w))x_i$. The intuition is that we want update the current weight vector w *towards* the direction of the attribute vector x_i of this positive tuple. For the latter case (i.e., $y_i = 0$), the impact of the given tuple on updating the weight vector w can be calculated as $w \leftarrow w - \eta P(\hat{y}_i = 1|x_i, w)x_i$. The intuition is that we want update the current weight vector w *away from* the direction of the attribute vector x_i of this negative tuple. From this perspective, the learning algorithm for training a logistic regression classifier bears some similarities to the perceptron algorithm. That is, both algorithms try to update the current weight vector w so that is (1) more aligned with the attribute vectors of positive tuples and (2) more mis-aligned with (i.e., towards the opposite direction of) the attribute vectors of negative tuples.

However, the two algorithms (perceptron vs. logistic regression) differ regarding to what extent the algorithms update the weight vector w . In perceptron, it uses a *fixed* learning rate η for all wrongly predicted tuples by the current weight vector w . On the other hand, in logistic regression, it depends on the learning rate η as well as $P(\hat{y}_i = 1|x_i, w)$ (i.e., the probability that the given tuple belongs to the positive class based on the current weight vector w). This makes the logistic regression algorithm *adaptive* in the following sense. For example, if $P(\hat{y}_i = 1|x_i, w)$ is high for a positive tuple, it means that the prediction by the current weight vector w for this positive tuple is not only correct (i.e., $P(\hat{y}_i = 1|x_i, w) > 0.5$), but also quite confident (i.e., $P(\hat{y}_i = 1|x_i, w)$ is close to 1). Then, the impact of this positive tuple (i.e., $\eta(1 - P(\hat{y}_i = 1|x_i, w))$) on updating the weight vector is relatively small. On the other hand, if $P(\hat{y}_i = 1|x_i, w)$ is high for a negative tuple, it means that the prediction by the current weight vector w for this negative tuple is either wrong (i.e., $P(\hat{y}_i = 1|x_i, w) > 0.5$), or correct but with low confidence (i.e., $P(\hat{y}_i = 1|x_i, w)$ is barely below 0.5). Then, the impact of this negative tuple (i.e., $\eta P(\hat{y}_i = 1|x_i, w)$) on updating the weight vector will be relatively large. In other words, the logistic regression learning algorithm pays more attention to those “hard” training tuples, which are either wrongly predicted or correctly predicted with a low confidence by the current weight vector w . Recall that for the example in Fig. 6.12(a), perceptron only uses x_1 and x_8 to update the current weight vector w since these two tuples are wrongly classified by the current w . In contrast, logistic regression uses *all* training tuples to update the weight vector w . Among them, x_1 and x_8 have the highest impact on updating w since they are both wrongly classified by the current classifier; x_2, x_3, x_5, x_9 and x_{10} have the least impact since they are all correctly classified by the current weight vector w with a high confidence; x_4, x_6 and x_7 have the moderate impact since they are correctly classified but with a relatively low confidence.

“How good is the logistic regression algorithm? What are the potential limitations and how to mitigate?” Since the log likelihood function $l(w)$ is a concave function, the algorithm for training a logistic regression classifier described above is guaranteed to converge to its optimal solution. However, if the training set is linearly separable, the algorithm might converge to a weight vector w with an infinitely large norm. (See an illustrative example in Fig. 6.14.) A “large” weight vector w could make the trained classifier prone to the noise of certain attributes of a given tuple. This will, in turn, lead to a poor generalization performance of the learned logistic regression classifier. In other words, the learned logistic regression classifier *overfits* the training set. An effective way to mitigate the overfitting is to introduce a regularization term $\|w\|_2^2$ into the objective function $l(w)$ to prevent the learned weight vector from

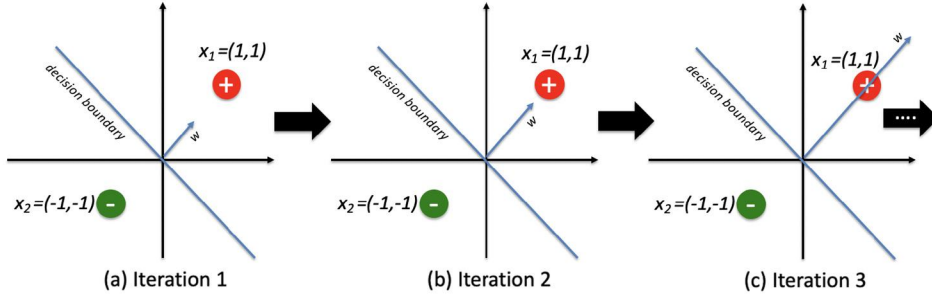


FIGURE 6.14

Illustration of the infinitely large weight vector of logistic regression in linearly separable case. There are two training tuples in 2d space, with one positive training tuple $x_1 = (1, 1)$ and one negative training tuple $x_2 = (-1, -1)$. For simplicity, we let the bias scalar $b = 0$ and the learning rate $\eta = 1$. Suppose that at iteration 1, the weight vector $w = (1, 1)$. Then, logistic regression algorithm introduced above will update the weight vector as $w_{\text{new}} = w_{\text{old}} + (1 - P(\hat{y}_1 = 1|x_1, w_{\text{old}}))x_1 - P(\hat{y}_2 = 1|x_2, w_{\text{old}})x_2 = (0.5, 0.5) + a(1, 1) = (1 + 2a)w_{\text{old}}$, where $a = 1 - P(\hat{y}_1 = 1|x_1, w_{\text{old}}) + P(\hat{y}_2 = 1|x_2, w_{\text{old}}) > 0$. As such, the new weight vector w_{new} shares the same direction as the old w_{old} . Therefore, the decision boundary remains the same, but new weight vector w_{new} grows in the magnitude by a factor of $(1 + 2a)$. This trend will continue as the logistic regression algorithm progresses, leading to a weight vector with an infinitely large magnitude.

becoming “too large.”¹¹ The second potential limitation is the *independence* assumption behind logistic regression. Recall that when we calculate the likelihood $L(w)$ of the training set, we simply multiply the likelihood of each training tuple together (Eq. (6.19)). This means that we have implicitly assumed that different training tuples are independent of each other. However, this assumption might be violated in some applications (e.g., users on a social network are interconnected with each other). The graph-based classification might provide a natural remedy for this issue. The third potential limitation lies in the computational challenge. Notice that in the updating rule $w \leftarrow w + \eta \sum_{i=1}^n (y_i - P(\hat{y}_i = 1|x_i, w))x_i$ described above, we need to calculate the gradients $(y_i - P(\hat{y}_i = 1|x_i, w))$ for *all* training tuples and then sum them up to update the weight vector w . If there are millions of training tuples, it is computationally very expensive to perform such computation. An efficient way to address this issue is to use *stochastic gradient descent* method to train a logistic regression classifier. That is, at each iteration, we will randomly sample a small subset of training tuples (this is often referred to as a *minibatch*) and *only* use the sampled tuples (instead of all training tuples) to update the weight vector. It is worth pointing out that the stochastic gradient descent is extensively used in many other data mining algorithms, such as deep learning methods, which will be introduced in Chapter 10.

¹¹ From the statistical parameter estimation perspective, we are switching from the maximum likelihood estimation (MLE) to maximum a posterior estimation (MAP). Adding a regularization term $\|w\|_2^2$ into $l(w)$ is equivalent to imposing a Gaussian prior with the mean vector at the origin for the weight vector w .